Data Cleaing

In [104]:

```
import pandas as pd
import numpy as np

# Load your CSV file into a pandas DataFrame
df = pd.read_csv('C:/Users/ritti/Downloads/VIT Downloads/CSE5007-Explor
df.head(3)
```

Out[104]:

	sr	state	District	person	Male	Female	growthinpopulation	rural	urban
0	1	ANdaman	District Nicobars (01), Andaman & Nicobar Isla...	314084	170319	143765	30.14	197886	116198
1	2	ANdaman	District Nicobars (02), Andaman & Nicobar Isla...	42068	22653	19415	7.19	42068	-
2	3	Andhra	District Adilabad (01), Andhra Pradesh (28)	2488003	1250958	1237045	19.06	1827986	NaN

3 rows × 29 columns

In [105]:
```
# df is our DataFrame and 'urban' is the column we want to drop
df = df.drop('urban', axis=1)
df.head(3)
```

Out[105]:

	sr	state	District	person	Male	Female	growthinpopulation	rural	urban.1
0	1	ANdaman	District Nicobars (01), Andaman & Nicobar Isla...	314084	170319	143765	30.14	197886	844.0
1	2	ANdaman	District Nicobars (02), Andaman & Nicobar Isla...	42068	22653	19415	7.19	42068	857.0
2	3	Andhra	District Adilabad (01), Andhra Pradesh (28)	2488003	1250958	1237045	19.06	1827986	989.0

3 rows × 28 columns

In [106]:

```
import pandas as pd

# df is our DataFrame and 'urban.1' is the column we want to rename
df = df.rename(columns={'urban.1': 'urban'})
df.head(3)
```

Out[106]:

	sr	state	District	person	Male	Female	growthinpopulation	rural	urban
0	1	ANdaman	District Nicobars (01), Andaman & Nicobar Isla...	314084	170319	143765	30.14	197886	844.0
1	2	ANdaman	District Nicobars (02), Andaman & Nicobar Isla...	42068	22653	19415	7.19	42068	857.0
2	3	Andhra	District Adilabad (01), Andhra Pradesh (28)	2488003	1250958	1237045	19.06	1827986	989.0

3 rows × 28 columns

In [107]:

```
# Print the number of duplicates, if any
print("Number of duplicate rows: ", df.duplicated().sum())

# Drop the duplicates
df = df.drop_duplicates()

# Print the number of duplicates again to confirm they've been dropped
print("Number of duplicate rows after dropping: ", df.duplicated().sum(
```

```
Number of duplicate rows:  0
Number of duplicate rows after dropping:  0
```

In [108]:
```
# Determine the number of missing values in each column
print("Number of missing values in each column:\n", df.isnull().sum())
```

```
Number of missing values in each column:
 sr                        0
state                      0
District                   0
person                     0
Male                       0
Female                     0
growthinpopulation         0
rural                      0
urban                      3
STPop                      3
Personlit_rate             0
Mlit_Rate                  0
Flit_Rate                  0
Totalworkers               0
Mainworkers                0
Marginal workers           0
Nonworkers                 0
Religion1                 53
religion1 population       0
Religion2                 53
Religion2 Population       0
religion3name             53
religion3poulation         0
inhabitadevillages        12
drinkingwater             12
electricity               12
primary school            12
Medical facility          12
dtype: int64
```

In [109]:
```
# Fill missing numeric values with the mean
numeric_columns = df.select_dtypes(include=[np.number]).columns
df[numeric_columns] = df[numeric_columns].apply(lambda x: x.fillna(x.me

# Fill missing non-numeric values
non_numeric_columns = df.select_dtypes(exclude=[np.number]).columns
df[non_numeric_columns] = df[non_numeric_columns].apply(lambda x: x.fil

df.to_csv('EDA1.csv', index=False)
```

In [110]:
```
1 print("Number of missing values in each column after filling:\n", df.is
```

```
Number of missing values in each column after filling:
 sr                      0
state                    0
District                 0
person                   0
Male                     0
Female                   0
growthinpopulation       0
rural                    0
urban                    0
STPop                    0
Personlit_rate           0
Mlit_Rate                0
Flit_Rate                0
Totalworkers             0
Mainworkers              0
Marginal workers         0
Nonworkers               0
Religion1                0
religion1 population     0
Religion2                0
Religion2 Population     0
religion3name            0
religion3poulation       0
inhabitadevillages       0
drinkingwater            0
electricity              0
primary school           0
Medical facility         0
dtype: int64
```

In [111]:
```
1 df.head(3)
```

Out[111]:

	sr	state	District	person	Male	Female	growthinpopulation	rural	urban
0	1	ANdaman	District Nicobars (01), Andaman & Nicobar Isla...	314084	170319	143765	30.14	197886	844.0
1	2	ANdaman	District Nicobars (02), Andaman & Nicobar Isla...	42068	22653	19415	7.19	42068	857.0
2	3	Andhra	District Adilabad (01), Andhra Pradesh (28)	2488003	1250958	1237045	19.06	1827986	989.0

3 rows × 28 columns

Fixing district names

In [112]:
```
import pandas as pd

# Read the CSV file as a data frame
df = pd.read_csv('C:/Users/ritti/Downloads/VIT Downloads/CSE5007-Explor

# Extract district names
df['District'] = df['District'].str.split('(').str[0].str.strip().str.s
# Save the modified data frame to a CSV file
df.to_csv('EDA2(District_name).csv', index=False)

```

In [113]:
```
df.head(3)
```

Out[113]:

	sr	state	District	person	Male	Female	growthinpopulation	rural	urban	S
0	1	ANdaman	Nicobars	314084	170319	143765	30.14	197886	844.0	
1	2	ANdaman	Nicobars	42068	22653	19415	7.19	42068	857.0	
2	3	Andhra	Adilabad	2488003	1250958	1237045	19.06	1827986	989.0	4

3 rows × 28 columns

Find outliers

In [114]:

```
from scipy import stats

# Calculate Z-scores
z_scores = stats.zscore(df.select_dtypes(include=['int', 'float']))

# Find outliers
outliers = df[(z_scores < -3) | (z_scores > 3)].any(axis=1)

# Print outliers
print("Outliers:",df[outliers])

```

```
Outliers:       sr             state       District   person     Male   Fe
male  \
6      7           Andhra      Hyderabad  3829753  1981173  1848580
14    15           Andhra  Visakhapatnam  3832336  1930197  1902139
17    18           Andhra         Guntur  4465144  2250279  2214865
27    28  ArunachalPradesh              *   122003    64184    57819
112  113      Chhattisgarh     Dantewada*   719487   356928   362559
..   ...               ...            ...      ...      ...      ...
582  583        WestBengal       Parganas  8934286  4638756  4295530
584  585        WestBengal        Bankura  3192695  1636002  1556693
585  586        WestBengal       Puruliya  2536516  1298078  1238438
586  587        WestBengal      Medinipur  9610788  4916370  4694418
589  590        WestBengal       Parganas  6906689  3564993  3341696

     growthinpopulation    rural   urban   STPop  ...  religion1 populatio
n  \
6                 17.18        -   933.0   34560  ...               212196
3
14                15.36  2301437   985.0  557572  ...               368828
0
17                 7.27  3179384   984.0  208157  ...               383420
4
27                67.21    59961   901.0   69007  ...                 5102
6
112               15.56   667475  1016.0  564931  ...                69651
6
..                  ...      ...     ...     ...  ...
...
582               22.64  4083339   926.0  198936  ...               672142
0
584               13.79  2957447   952.0  330783  ...               269302
2
585               13.96  2281090   954.0  463452  ...               211603
7
586               15.68  8626883   955.0  798684  ...               822477
9
589               20.89  5820469   937.0   84766  ...               454845
9

        Religion2  Religion2 Population  religion3name  religion3poulation
\
6       2.Muslims              1576583   3.Christians              92915
14      2.Muslims                71669   3.Christians              62227
17      2.Muslims               487839   3.Christians             131713
27   2.Christians                36574       3.Others              25395
112      2.Others                 8644   3.Christians               6213
..            ...                  ...            ...                ...
582     2.Muslims              2164058   3.Christians              20138
584      2.Others               251401      3.Muslims             239722
585      2.Others               226912      3.Muslims             180694
586     2.Muslims              1088618       3.Others             262578
589     2.Muslims              2295967   3.Christians              52835

     inhabitadevillages  drinkingwater  electricity  primary school  \
6           1016.451557     1009.67128   774.759516      796.794118
14          3108.000000     3091.00000  3098.000000     2293.000000
17           694.000000      694.00000   694.000000      690.000000
27           267.000000      267.00000    60.000000       69.000000
112         1220.000000     1204.00000   578.000000     1006.000000
..                  ...            ...          ...             ...
582         1572.000000     1567.00000  1274.000000     1391.000000
```

```
584          3577.000000     3548.00000  2214.000000     2663.000000
585          2468.000000     2452.00000  1232.000000     2135.000000
586         10548.000000    10475.00000  4835.000000     6133.000000
589          2087.000000     2079.00000  1322.000000     1774.000000

    Medical facility
6                118
14              1647
17               584
27                17
112              201
..               ...
582              955
584             1081
585             1173
586             3560
589             1311

[65 rows x 28 columns]
```

The output you're seeing is a DataFrame that contains all the rows in your original dataset that were identified as outliers. These are the data points that significantly differ from other observations in your dataset.

The method you've used to detect outliers is likely based on the Z-score or IQR method. The Z-score method identifies an observation as an outlier if its Z-score (a measure of how many standard deviations an element is from the mean) is less than -3 or greater than 3. This means that the data points identified as outliers are those that are exceptionally low or high compared to the rest of the data.

In the output DataFrame, each row represents a district with various statistics such as population by gender, growth in population, literacy rate by gender, worker classification, and religious demographics. The districts listed in this DataFrame are those that have at least one statistic that is considered an outlier.

Remember, while outliers can skew your data, it's important to investigate why they exist before deciding to remove them. They could be due to errors, but they could also be important, valid observations that indicate something significant about the phenomenon you're studying.

Dimensionality reduction

Dimensionality reduction is a technique used in machine learning to reduce the number of input variables in a dataset. High-dimensional data can be difficult to work with for several reasons. For example, it can lead to overfitting, it can be computationally expensive, and it can make visualizing the data challenging.

There are several methods for dimensionality reduction, including:

1. **Principal Component Analysis (PCA)**: This is a technique that transforms the data into a new coordinate system such that the greatest variance by any projection of the data comes to lie on the first coordinate (called the first principal component), the second greatest variance on the second coordinate, and so on.

2. **Linear Discriminant Analysis (LDA)**: This is a method used in statistics, pattern recognition, and machine learning to find a linear combination of features that characterizes or separates two or more classes of objects or events.
3. **t-Distributed Stochastic Neighbor Embedding (t-SNE)**: This is a technique for dimensionality reduction that is particularly well suited for the visualization of high-dimensional datasets.
4. **Autoencoders**: These are a type of artificial neural network used for learning efficient codings of input data. They can be used for dimensionality reduction.
5. **Feature selection methods**: These include techniques like backward elimination, forward selection, and bidirectional elimination.

Each of these methods has its own strengths and weaknesses, and is suitable for different

Dimensionality reduction using Principal Component Analysis (PCA) with Python's sklearn library. PCA is a commonly used method for dimensionality reduction that transforms the original variables to a new set of variables, which are linear combinations of the original variables.

In [115]:
```
1
2  from sklearn.decomposition import PCA
3
4  # Select numeric columns
5  numeric_cols = df.select_dtypes(include=[np.number]).columns
6
7  # Normalize the numeric features
8  df_normalized = df.copy()
9  df_normalized[numeric_cols] = (df[numeric_cols] - df[numeric_cols].mean
10
11 # Fit and transform the data
12 pca = PCA(n_components=10)
13 df_pca = pca.fit_transform(df_normalized[numeric_cols])
14
15 # Convert the transformed data into a DataFrame
16 df_pca = pd.DataFrame(df_pca, columns=['PC' + str(i) for i in range(1,
17
18 print(df_pca.head())
19
```

```
        PC1       PC2       PC3       PC4       PC5       PC6       PC7  \
0 -3.763857  2.077311 -0.158831  1.132385  1.947819 -0.050860  0.866245
1 -4.403762  1.136349 -0.021499 -0.500638  1.167866 -0.826715  1.413649
2  2.369451 -1.684128  0.376262 -1.335462  1.582645 -0.485059 -0.030763
3  1.254521 -0.850418 -0.744616 -2.381085  0.852039 -0.791409 -0.134390
4  3.635922 -0.343614 -1.067277 -2.245313  1.313231 -1.463458 -0.920197

        PC8       PC9      PC10
0 -0.073075  0.393618  0.049192
1 -1.102188  0.367812  0.144878
2  0.410876 -0.767453 -0.327471
3  0.626847 -0.463954 -0.278215
4  0.188523 -0.482735 -0.058430
```

The output you're seeing is the result of Principal Component Analysis (PCA), a technique used for dimensionality reduction. The goal of PCA is to create new features (the principal components) that maintain as much information as possible from the original data while reducing the dimensionality.

In our output:

- Each column labeled `PC1`, `PC2`, `PC3`, etc., represents a principal component. These are the new features created by PCA. They are linear combinations of your original features, chosen in such a way that each new feature is orthogonal (uncorrelated) to the ones before it, and that each feature explains as much of the remaining variance in your data as possible.
- Each row represents an observation (a district in your case) in your dataset, but in the transformed feature space.
- The numbers you see are the coordinates of your original data points (districts) along each of these new features. For example, the `-3.754562` under `PC1` for the first district means that this district is located at `-3.754562` along the `PC1` dimension.

Remember, the principal components are less interpretable than the original features, because they are combinations of all original features. However, they allow you to reduce the dimensionality of your data while keeping as much variance (information) as possible.

Merging the two datasets

In [116]:

```
# Read both datasets
dataset1 = pd.read_csv("C:/Users/ritti/Downloads/VIT Downloads/CSE5007-
dataset2 = pd.read_csv("C:/Users/ritti/Downloads/VIT Downloads/CSE5007-

# Convert 'state' and 'District' columns to lower case
dataset1['state'] = dataset1['state'].str.lower()
dataset1['District'] = dataset1['District'].str.lower()
dataset2['state'] = dataset2['state'].str.lower()
dataset2['District'] = dataset2['District'].str.lower()

# Merge datasets on 'state' and 'District'
merged_dataset = pd.merge(dataset1, dataset2, on=['state', 'District'])

# Print the merged dataset
print(merged_dataset)

# Write the merged dataset to a new CSV file
merged_dataset.to_csv("EDA3(Merged_dataset).csv", index=False)


```

```
      state    District    YEAR MURDER ATTEMPT TO MURDER RAPE CUSTODIAL RA
PE  \
0     assam     barpeta  2001.0     64                5   28
0
1     assam  bongaigaon  2001.0     45               22   20
0
2     assam      cachar  2001.0     52               30   45
0
3     assam     darrang  2001.0     61               13   48
0
4     assam     dhemaji  2001.0     15               16   42
0
...     ...         ...     ...    ...              ...  ...
...
2659  delhi       north  2020.0      0                0   47
0
2660  delhi       south  2020.0      0                0   74
0
2661  delhi        west  2020.0      1                0   56
0
2662  delhi        west  2020.0      1                0   56
0
2663  delhi        west  2020.0      1                0   56
0

     OTHER RAPE KIDNAPPING & ABDUCTION  \
0            28                    105
1            20                     36
2            45                    104
3            48                     64
4            42                     31
...         ...                    ...
2659          0                    109
2660          0                    176
2661          0                    189
2662          0                    189
2663          0                    189

     KIDNAPPING AND ABDUCTION OF WOMEN AND GIRLS  ... religion1 population
\
0                                             88  ...              977943
1                                             21  ...              535464
2                                             74  ...              886761
3                                             47  ...              868532
4                                             20  ...              548780
...                                          ...  ...                 ...
2659                                         150  ...              609493
2660                                         183  ...             1824645
2661                                         217  ...             2540491
2662                                         217  ...             1745810
2663                                         217  ...             1612169

      Religion2 Religion2 Population religion3name religion3poulation  \
0     2.Muslims               662066  3.Christians               5267
1     2.Muslims               348573  3.Christians              18728
2     2.Muslims               522051  3.Christians              31306
3     2.Muslims               534658  3.Christians              97306
4     2.Muslims                10533  3.Christians               6390
...         ...                  ...          ...                ...
2659  2.Muslims               126093       3.Sikhs              20819
2660  2.Muslims               314015       3.Sikhs              75102
```

```
2661  2.Muslims              173409       3.Sikhs              91723
2662    2.Sikhs              247312     3.Muslims             107079
2663  2.Muslims               76429       3.Sikhs              29470

      inhabitadevillages  drinkingwater  electricity  primary school  \
0                 1050.0         1050.0        775.0           967.0
1                  881.0          881.0        607.0           805.0
2                 1020.0         1020.0        741.0           902.0
3                 1319.0         1319.0       1032.0          1246.0
4                 1236.0         1236.0        253.0          1068.0
...                  ...            ...          ...             ...
2659                 5.0            5.0          4.0             3.0
2660                16.0           16.0         16.0            12.0
2661                62.0           62.0         62.0            57.0
2662                 9.0            9.0          9.0             9.0
2663                51.0           51.0         51.0            49.0

      Medical facility
0                  430
1                  112
2                  134
3                  356
4                   83
...                ...
2659                 3
2660                 7
2661                44
2662                 5
2663                26

[2664 rows x 42 columns]
```

In [118]:

```
# merged_dataset is our DataFrame and 'sr' is the column we want to dro
new_merged_dataset =merged_dataset.drop('sr', axis=1)
new_merged_dataset.to_csv("C:/Users/ritti/Downloads/VIT Downloads/CSE50
new_merged_dataset.head(3)
```

Out[118]:

	state	District	YEAR	MURDER	ATTEMPT TO MURDER	RAPE	CUSTODIAL RAPE	OTHER RAPE	KIDNAPPING & ABDUCTION
0	assam	barpeta	2001.0	64	5	28	0	28	105
1	assam	bongaigaon	2001.0	45	22	20	0	20	36
2	assam	cachar	2001.0	52	30	45	0	45	104

3 rows × 41 columns

In [119]:

```
# Check where '-' values are in the DataFrame
mask = new_merged_dataset.isin(['-'])

# Print the number of '-' values in each column
print(mask.sum())
mask.head(3)
```

```
state                                                  0
District                                               0
YEAR                                                   0
MURDER                                                 0
ATTEMPT TO MURDER                                      0
RAPE                                                   0
CUSTODIAL RAPE                                         0
OTHER RAPE                                             0
KIDNAPPING & ABDUCTION                                 0
KIDNAPPING AND ABDUCTION OF WOMEN AND GIRLS            0
KIDNAPPING AND ABDUCTION OF OTHERS                     0
DOWRY DEATHS                                           0
ASSAULT ON WOMEN WITH INTENT TO OUTRAGE HER MODESTY     0
INSULT TO MODESTY OF WOMEN                             0
CRUELTY BY HUSBAND OR HIS RELATIVES                    0
IMPORTATION OF GIRLS FROM FOREIGN COUNTRIES            0
person                                                 0
Male                                                   0
Female                                                 0
growthinpopulation                                     0
rural                                                 11
urban                                                  0
STPop                                                364
Personlit_rate                                         0
Mlit_Rate                                              0
Flit_Rate                                              0
Totalworkers                                           0
Mainworkers                                            0
Marginal workers                                       0
Nonworkers                                             0
Religion1                                              0
religion1 population                                   0
Religion2                                              0
Religion2 Population                                   0
religion3name                                          0
religion3poulation                                     0
inhabitadevillages                                     0
drinkingwater                                          0
electricity                                            0
primary school                                         0
Medical facility                                       1
dtype: int64
```

Out[119]:

	state	District	YEAR	MURDER	ATTEMPT TO MURDER	RAPE	CUSTODIAL RAPE	OTHER RAPE	KIDNAPPING & ABDUCTION	KI AE O A
0	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	

3 rows × 41 columns

In this below code:

1. We first convert all columns to numeric using pd.to_numeric() function.
2. Then, we replace '-' with NaN.
3. Finally, we fill missing numeric values with the mean of each column using fillna() method.

In [120]:

```
# Convert numeric columns to numeric
numeric_columns = new_merged_dataset.select_dtypes(include=np.number).c
new_merged_dataset_numeric = new_merged_dataset[numeric_columns].apply(

# Fill missing numeric values with mean
new_merged_dataset_numeric.fillna(new_merged_dataset_numeric.mean(), in

# Fill "-" values in non-numeric columns with a placeholder
non_numeric_columns = new_merged_dataset.select_dtypes(exclude=np.numbe
new_merged_dataset_non_numeric = new_merged_dataset[non_numeric_columns

# Concatenate numeric and non-numeric columns
merged_dataset_filled = pd.concat([new_merged_dataset_numeric, new_merg

# Print the number of missing values in each column after filling
print(merged_dataset_filled.isnull().sum())

```

```
YEAR                                                  0
person                                                0
Male                                                  0
Female                                                0
growthinpopulation                                    0
urban                                                 0
Personlit_rate                                        0
Mlit_Rate                                             0
Flit_Rate                                             0
Totalworkers                                          0
Mainworkers                                           0
Marginal workers                                      0
Nonworkers                                            0
religion1 population                                  0
Religion2 Population                                  0
religion3poulation                                    0
inhabitadevillages                                    0
drinkingwater                                         0
electricity                                           0
primary school                                        0
state                                                 0
District                                              0
MURDER                                              169
ATTEMPT TO MURDER                                     0
RAPE                                                  0
CUSTODIAL RAPE                                        2
OTHER RAPE                                          169
KIDNAPPING & ABDUCTION                                0
KIDNAPPING AND ABDUCTION OF WOMEN AND GIRLS           0
KIDNAPPING AND ABDUCTION OF OTHERS                    0
DOWRY DEATHS                                          0
ASSAULT ON WOMEN WITH INTENT TO OUTRAGE HER MODESTY    0
INSULT TO MODESTY OF WOMEN                            0
CRUELTY BY HUSBAND OR HIS RELATIVES                   0
IMPORTATION OF GIRLS FROM FOREIGN COUNTRIES           0
rural                                                11
STPop                                               364
Religion1                                             0
Religion2                                             0
religion3name                                         0
Medical facility                                      1
dtype: int64
```

1. Fill missing values in MURDER, CUSTODIAL RAPE, and OTHER RAPE columns with the mean of each respective column.
2. Fill missing values in rural and Medical facility columns with the mode (most common value) of each column.
3. Drop the STPop column.

In [124]:

```
your DataFrame

c

ic = merged_dataset_filled.apply(pd.to_numeric, errors='ignore')

RDER, CUSTODIAL RAPE, and OTHER RAPE columns with the mean
, 'CUSTODIAL RAPE', 'OTHER RAPE']
ic[numeric_columns] = merged_dataset_filled_numeric[numeric_columns].fillna(

ral and Medical facility columns with mode
ic['rural'].fillna(merged_dataset_filled_numeric['rural'].mode()[0], inplace
ic['Medical facility'].fillna(merged_dataset_filled_numeric['Medical facilit


ic.drop('STPop', axis=1, inplace=True)

ic.to_csv("C:/Users/ritti/Downloads/VIT Downloads/CSE5007-Exploratory Data A
district position got changed


ming missing values
_numeric.isnull().sum())

```

```
YEAR                                                  0
person                                                0
Male                                                  0
Female                                                0
growthinpopulation                                    0
urban                                                 0
Personlit_rate                                        0
Mlit_Rate                                             0
Flit_Rate                                             0
Totalworkers                                          0
Mainworkers                                           0
Marginal workers                                      0
Nonworkers                                            0
religion1 population                                  0
Religion2 Population                                  0
religion3poulation                                    0
inhabitadevillages                                    0
drinkingwater                                         0
electricity                                           0
primary school                                        0
state                                                 0
District                                              0
MURDER                                                0
ATTEMPT TO MURDER                                     0
RAPE                                                  0
CUSTODIAL RAPE                                        0
OTHER RAPE                                            0
KIDNAPPING & ABDUCTION                                0
KIDNAPPING AND ABDUCTION OF WOMEN AND GIRLS           0
KIDNAPPING AND ABDUCTION OF OTHERS                    0
DOWRY DEATHS                                          0
ASSAULT ON WOMEN WITH INTENT TO OUTRAGE HER MODESTY    0
INSULT TO MODESTY OF WOMEN                            0
CRUELTY BY HUSBAND OR HIS RELATIVES                   0
IMPORTATION OF GIRLS FROM FOREIGN COUNTRIES           0
rural                                                 0
Religion1                                             0
Religion2                                             0
religion3name                                         0
Medical facility                                      0
dtype: int64
```

```
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/2914594166.py:4: FutureWa
rning: errors='ignore' is deprecated and will raise in a future version. U
se to_numeric without passing `errors` and catch exceptions explicitly ins
tead
  merged_dataset_filled_numeric = merged_dataset_filled.apply(pd.to_numeri
c, errors='ignore')
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/2914594166.py:11: FutureW
arning: A value is trying to be set on a copy of a DataFrame or Series thr
ough chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never wor
k because the intermediate object on which we are setting values always be
haves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  merged_dataset_filled_numeric['rural'].fillna(merged_dataset_filled_nume
ric['rural'].mode()[0], inplace=True)
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/2914594166.py:12: FutureW
arning: A value is trying to be set on a copy of a DataFrame or Series thr
ough chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never wor
k because the intermediate object on which we are setting values always be
haves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  merged_dataset_filled_numeric['Medical facility'].fillna(merged_dataset_
filled_numeric['Medical facility'].mode()[0], inplace=True)
```

Visualize and Analyze

future work:-

1. Population Analysis:

- Bar plot showing the population distribution across states.
- Bar plot showing the population distribution across districts within a state.

2. Crime Analysis:

- Bar plots showing various types of crimes across states.
- Bar plots showing various types of crimes across districts within a state.

3. Education Analysis:

- Scatter plot showing literacy rate vs. population for each state.
- Scatter plot showing literacy rate vs. population for each district within a state.

4. Infrastructure Analysis:

- Bar plot showing the availability of amenities (e.g., drinking water, electricity) across states.

- Bar plot showing the availability of amenities across districts within a state.

5. Workforce Analysis:

- Pie chart showing the distribution of workers (main workers, marginal workers, non-workers) across states.
- Pie chart showing the distribution of workers across districts within a state.

6. Religion Analysis:

- Stacked bar plot showing the population distribution of different religions across states.
- Stacked bar plot showing the population distribution of different religions across districts within a state.

In [146]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming the data is stored in a CSV file named 'data.csv'
dfv = pd.read_csv("C:/Users/ritti/Downloads/VIT Downloads/CSE5007-Explo

# Basic info about the dataset
print(dfv.info())

```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2664 entries, 0 to 2663
Data columns (total 40 columns):
 #   Column                                                Non-Null Count
Dtype
---  ------                                                --------------
-----
 0   YEAR                                                  2664 non-null
float64
 1   person                                                2664 non-null
int64
 2   Male                                                  2664 non-null
int64
 3   Female                                                2664 non-null
int64
 4   growthinpopulation                                    2664 non-null
float64
 5   urban                                                 2664 non-null
float64
 6   Personlit_rate                                        2664 non-null
float64
 7   Mlit_Rate                                             2664 non-null
float64
 8   Flit_Rate                                             2664 non-null
float64
 9   Totalworkers                                          2664 non-null
int64
 10  Mainworkers                                           2664 non-null
int64
 11  Marginal workers                                      2664 non-null
int64
 12  Nonworkers                                            2664 non-null
int64
 13  religion1 population                                  2664 non-null
int64
 14  Religion2 Population                                  2664 non-null
int64
 15  religion3poulation                                    2664 non-null
int64
 16  inhabitadevillages                                    2664 non-null
float64
 17  drinkingwater                                         2664 non-null
float64
 18  electricity                                           2664 non-null
float64
 19  primary school                                        2664 non-null
float64
 20  state                                                 2664 non-null
object
 21  District                                              2664 non-null
object
 22  MURDER                                                2664 non-null
float64
 23  ATTEMPT TO MURDER                                     2664 non-null
int64
 24  RAPE                                                  2664 non-null
int64
 25  CUSTODIAL RAPE                                        2664 non-null
float64
 26  OTHER RAPE                                            2664 non-null
float64
```

```
 27  KIDNAPPING & ABDUCTION                                2664 non-null
int64
 28  KIDNAPPING AND ABDUCTION OF WOMEN AND GIRLS           2664 non-null
int64
 29  KIDNAPPING AND ABDUCTION OF OTHERS                    2664 non-null
int64
 30  DOWRY DEATHS                                          2664 non-null
int64
 31  ASSAULT ON WOMEN WITH INTENT TO OUTRAGE HER MODESTY   2664 non-null
int64
 32  INSULT TO MODESTY OF WOMEN                            2664 non-null
int64
 33  CRUELTY BY HUSBAND OR HIS RELATIVES                   2664 non-null
int64
 34  IMPORTATION OF GIRLS FROM FOREIGN COUNTRIES           2664 non-null
int64
 35  rural                                                 2664 non-null
float64
 36  Religion1                                             2664 non-null
object
 37  Religion2                                             2664 non-null
object
 38  religion3name                                         2664 non-null
object
 39  Medical facility                                      2664 non-null
float64
dtypes: float64(15), int64(20), object(5)
memory usage: 832.6+ KB
None
```

In [147]:

```
# Summary statistics
dfv.describe()
```

Out[147]:

	YEAR	person	Male	Female	growthinpopulation	urba
count	2664.000000	2.664000e+03	2.664000e+03	2.664000e+03	2664.000000	2664.0000
mean	2008.643769	1.643346e+06	8.487270e+05	7.946194e+05	23.934009	934.1610
std	5.595346	9.919541e+05	5.155389e+05	4.782411e+05	12.515223	55.6237
min	2001.000000	4.103000e+04	2.341400e+04	1.761600e+04	-1.910000	752.0000
25%	2004.000000	9.702940e+05	5.170150e+05	4.714340e+05	17.080000	909.0000
50%	2008.000000	1.515148e+06	7.849460e+05	7.313510e+05	22.390000	935.0000
75%	2012.000000	2.277475e+06	1.172753e+06	1.109682e+06	27.370000	957.0000
max	2020.000000	8.640419e+06	4.741720e+06	3.898699e+06	95.010000	1136.0000

8 rows × 35 columns

In [148]:
```
# Checking for missing values
print(dfv.isnull().sum())
```

```
YEAR                                                0
person                                              0
Male                                                0
Female                                              0
growthinpopulation                                  0
urban                                               0
Personlit_rate                                      0
Mlit_Rate                                           0
Flit_Rate                                           0
Totalworkers                                        0
Mainworkers                                         0
Marginal workers                                    0
Nonworkers                                          0
religion1 population                                0
Religion2 Population                                0
religion3poulation                                  0
inhabitadevillages                                  0
drinkingwater                                       0
electricity                                         0
primary school                                      0
state                                               0
District                                            0
MURDER                                              0
ATTEMPT TO MURDER                                   0
RAPE                                                0
CUSTODIAL RAPE                                      0
OTHER RAPE                                          0
KIDNAPPING & ABDUCTION                              0
KIDNAPPING AND ABDUCTION OF WOMEN AND GIRLS         0
KIDNAPPING AND ABDUCTION OF OTHERS                  0
DOWRY DEATHS                                        0
ASSAULT ON WOMEN WITH INTENT TO OUTRAGE HER MODESTY  0
INSULT TO MODESTY OF WOMEN                          0
CRUELTY BY HUSBAND OR HIS RELATIVES                 0
IMPORTATION OF GIRLS FROM FOREIGN COUNTRIES         0
rural                                               0
Religion1                                           0
Religion2                                           0
religion3name                                       0
Medical facility                                    0
dtype: int64
```

In [149]:
```
1 dfv.head()
```

Out[149]:

	YEAR	person	Male	Female	growthinpopulation	urban	Personlit_rate	Mlit_Rate	Flit
0	2001.0	1647201	848578	798623	18.53	941.0	56.24	64.83	
1	2001.0	904835	465240	439595	12.23	945.0	59.33	67.67	
2	2001.0	1444921	743042	701879	18.66	945.0	67.82	75.73	
3	2001.0	1504320	773861	730459	15.79	944.0	55.44	63.91	
4	2001.0	571944	294643	277301	18.93	941.0	64.48	74.41	

5 rows × 40 columns

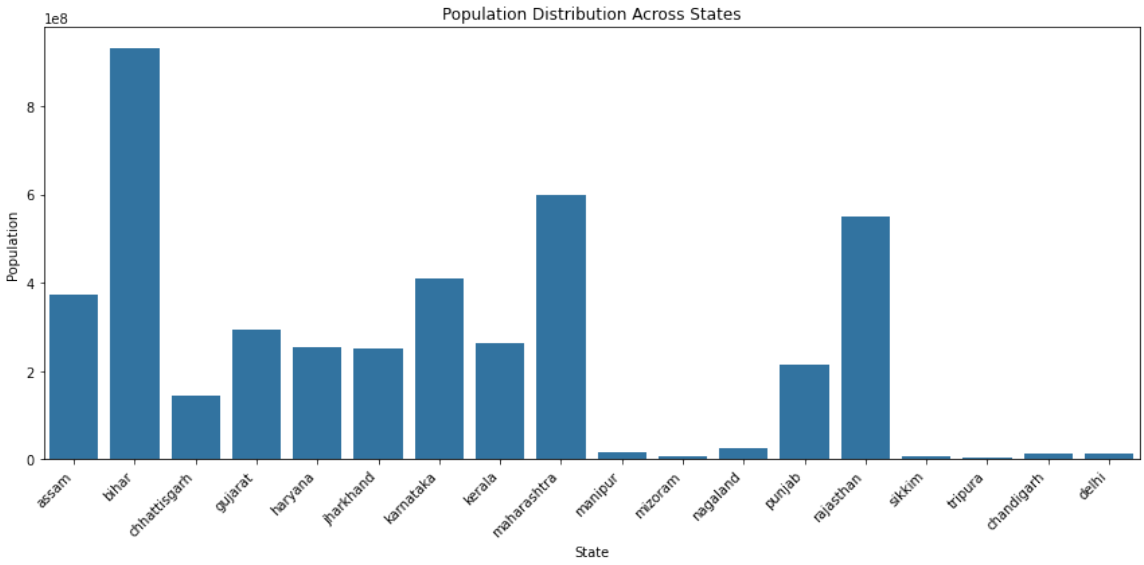
1. Population Analysis:

- Bar plot showing the population distribution across states.
- Bar plot showing the population distribution across districts within a state.

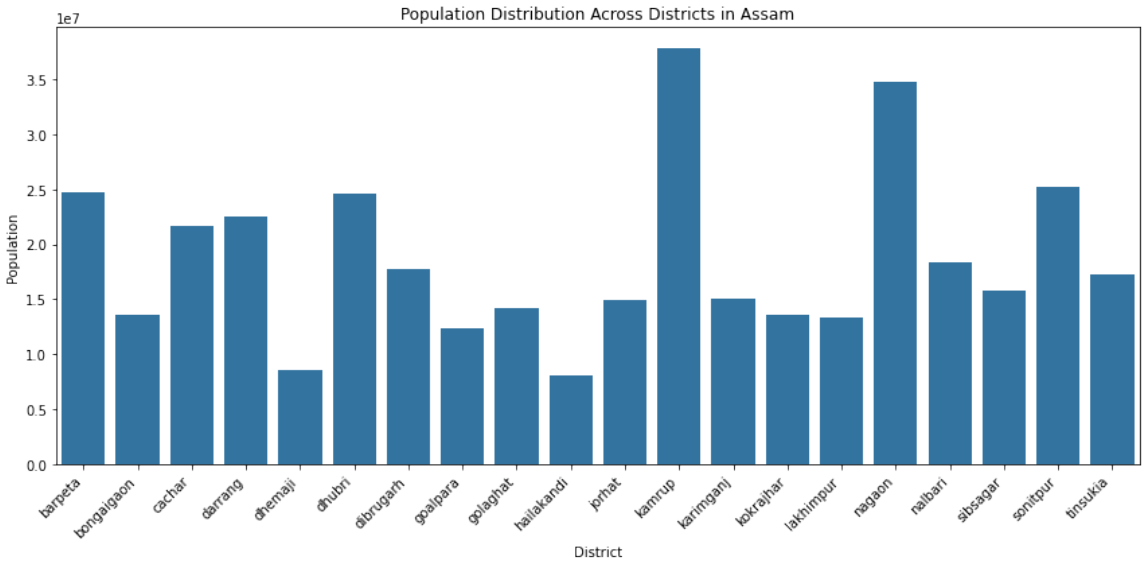
In [150]:

```
# Population distribution across states
plt.figure(figsize=(12, 6))
sns.barplot(x='state', y='person', data=dfv, estimator=sum, ci=None)
plt.title('Population Distribution Across States')
plt.xlabel('State')
plt.ylabel('Population')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Population distribution across districts within a state (for a specif
state_name = 'assam'  # Replace with the state you want to visualize
plt.figure(figsize=(12, 6))
sns.barplot(x='District', y='person', data=dfv[dfv['state'].str.lower()
plt.title(f'Population Distribution Across Districts in {state_name.cap
plt.xlabel('District')
plt.ylabel('Population')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

```
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/952512567.py:3: FutureWar
ning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.barplot(x='state', y='person', data=dfv, estimator=sum, ci=None)
```




```
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/952512567.py:14: FutureWa
rning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.barplot(x='District', y='person', data=dfv[dfv['state'].str.lower()
== state_name], estimator=sum, ci=None)
```

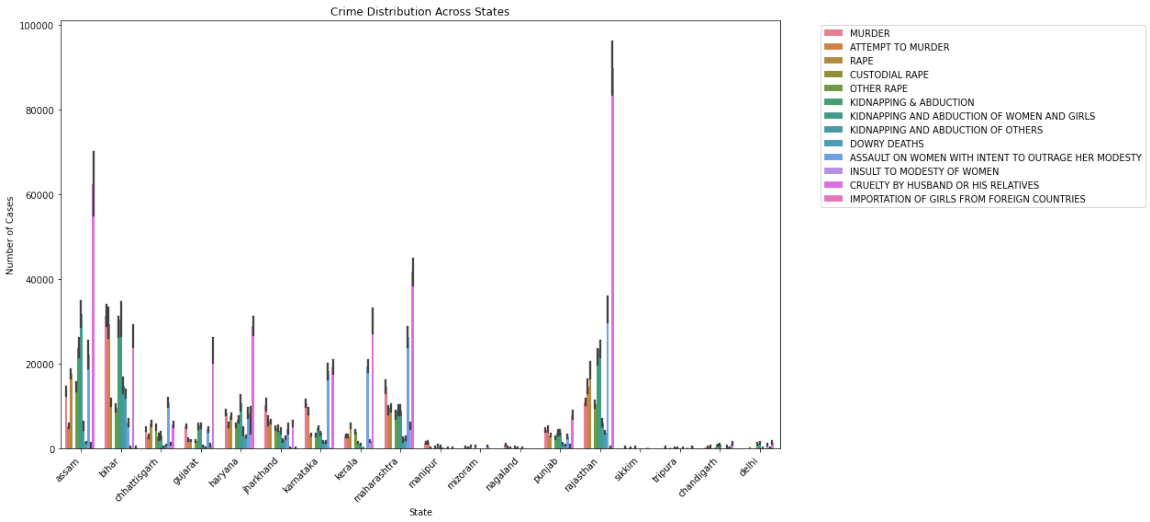


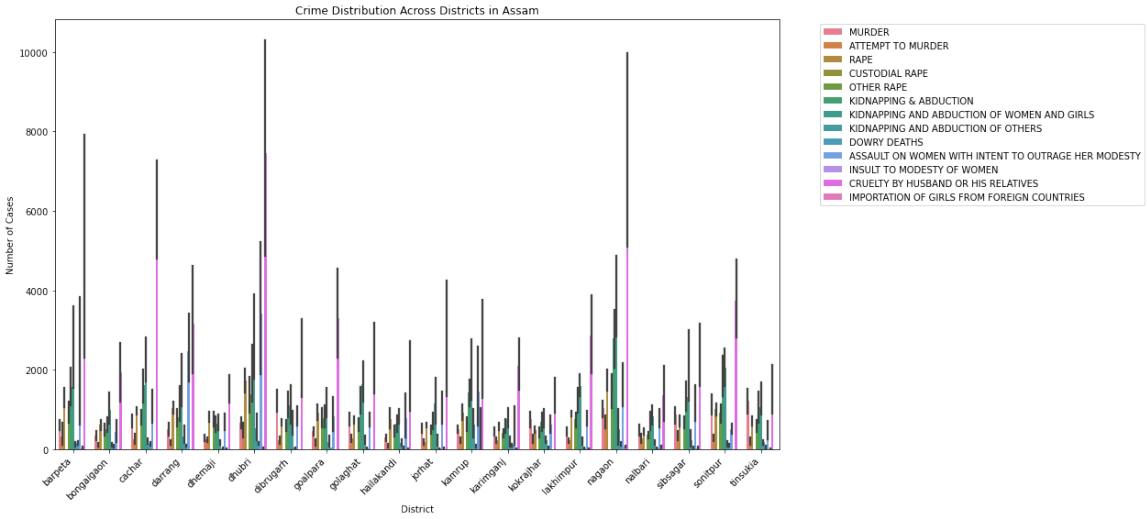
2. Crime Analysis:

- Bar plots showing various types of crimes across states.
- Bar plots showing various types of crimes across districts within a state.

In [156]:

```
# Adjust the crime_columns list to exclude the 'YEAR' column
crime_columns = ['MURDER', 'ATTEMPT TO MURDER', 'RAPE',
                 'CUSTODIAL RAPE', 'OTHER RAPE', 'KIDNAPPING & ABDUCTIO
                 'KIDNAPPING AND ABDUCTION OF WOMEN AND GIRLS',
                 'KIDNAPPING AND ABDUCTION OF OTHERS', 'DOWRY DEATHS',
                 'ASSAULT ON WOMEN WITH INTENT TO OUTRAGE HER MODESTY',
                 'INSULT TO MODESTY OF WOMEN', 'CRUELTY BY HUSBAND OR H
                 'IMPORTATION OF GIRLS FROM FOREIGN COUNTRIES']
# Remove the 'YEAR' column from the DataFrame
#dfv = dfv.drop(columns=['YEAR'])


# Melt the DataFrame to reshape it
melted_df = dfv.melt(id_vars=['state', 'District'], value_vars=crime_co

# Create bar plot showing various types of crimes across states
plt.figure(figsize=(12, 8))
sns.barplot(data=melted_df, x='state', y='Number of Cases', hue='Crime'
plt.title('Crime Distribution Across States')
plt.xlabel('State')
plt.ylabel('Number of Cases')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()

# Bar plots showing various types of crimes across districts within a s
state_name = 'assam'  # Replace with the state you want to visualize
plt.figure(figsize=(12, 8))
sns.barplot(data=melted_df[melted_df['state'].str.lower() == state_name
plt.title(f'Crime Distribution Across Districts in {state_name.capitali
plt.xlabel('District')
plt.ylabel('Number of Cases')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()

```



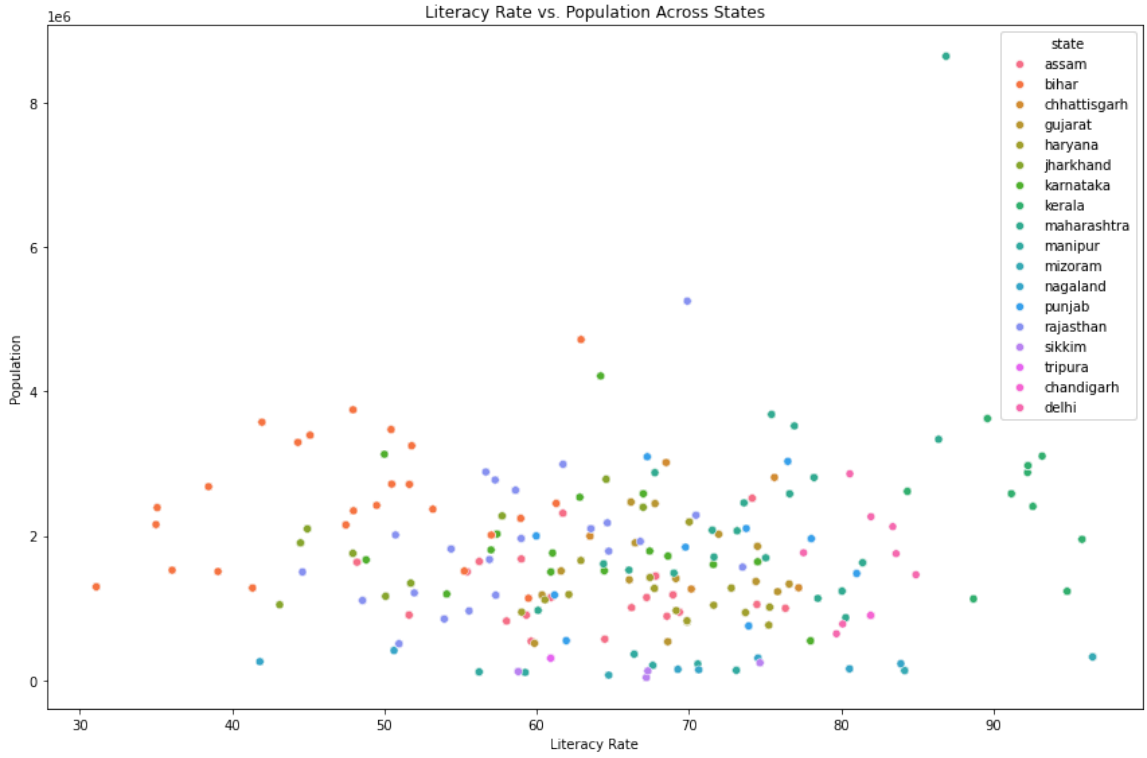


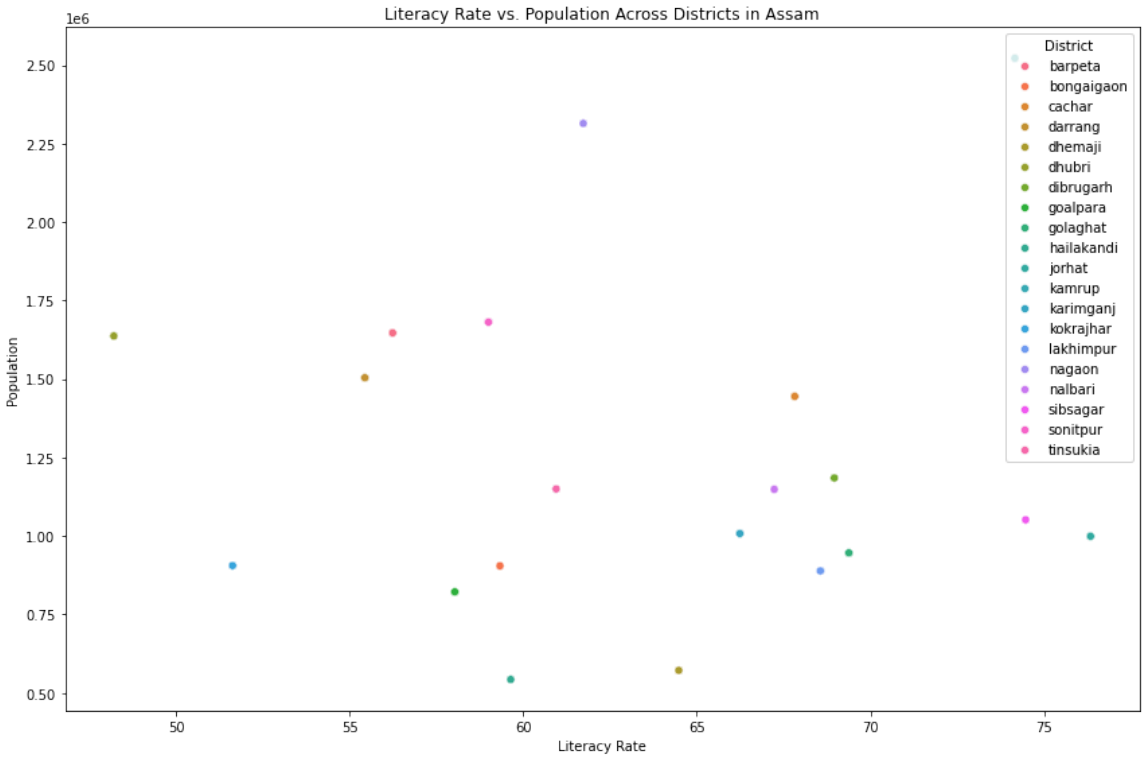
3. Education Analysis:

- Scatter plot showing literacy rate vs. population for each state.
- Scatter plot showing literacy rate vs. population for each district within a state.

In [161]:

```
# Scatter plot showing literacy rate vs. population for each state
plt.figure(figsize=(12, 8))
sns.scatterplot(data=dfv, x='Personlit_rate', y='person', hue='state',
plt.title('Literacy Rate vs. Population Across States')
plt.xlabel('Literacy Rate')
plt.ylabel('Population')
plt.tight_layout()
plt.show()


# Scatter plot showing literacy rate vs. population for each district w
state_name = 'assam'  # Replace with the state you want to visualize
plt.figure(figsize=(12, 8))
sns.scatterplot(data=dfv[dfv['state'].str.lower() == state_name], x='Pe
plt.title(f'Literacy Rate vs. Population Across Districts in {state_nam
plt.xlabel('Literacy Rate')
plt.ylabel('Population')
plt.tight_layout()
plt.show()


```





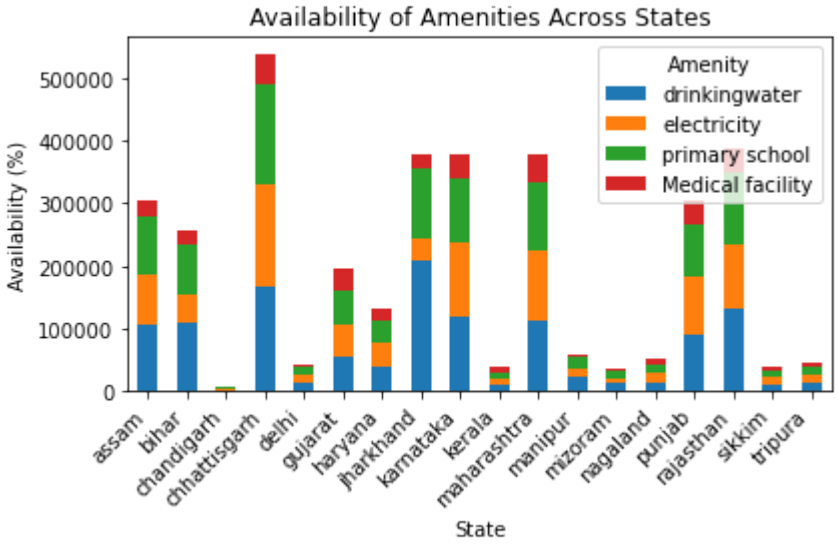
4. Infrastructure Analysis:

- Bar plot showing the availability of amenities (e.g., drinking water, electricity) across states.
- Bar plot showing the availability of amenities across districts within a state.(All states for a particular state)

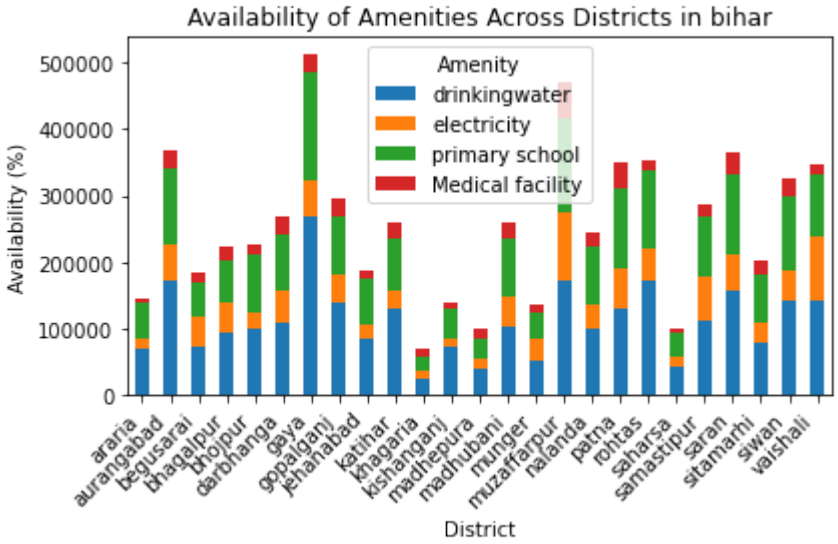
In [165]:

```
# Calculate the percentage of availability of amenities across states
amenities = ['drinkingwater', 'electricity', 'primary school', 'Medical
amenity_availability_states = dfv.groupby('state')[amenities].mean() *

# Plotting bar plot showing the availability of amenities across states
plt.figure(figsize=(12, 8))
amenity_availability_states.plot(kind='bar', stacked=True)
plt.title('Availability of Amenities Across States')
plt.xlabel('State')
plt.ylabel('Availability (%)')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Amenity')
plt.tight_layout()
plt.show()

# Bar plot showing the availability of amenities across districts withi
state_name = 'bihar'  # Replace with the state you want to visualize
amenity_availability_districts = dfv[dfv['state'] == state_name].groupb

# Plotting bar plot showing the availability of amenities across distri
plt.figure(figsize=(12, 8))
amenity_availability_districts.plot(kind='bar', stacked=True)
plt.title(f'Availability of Amenities Across Districts in {state_name}'
plt.xlabel('District')
plt.ylabel('Availability (%)')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Amenity')
plt.tight_layout()
plt.show()

```

```
<Figure size 864x576 with 0 Axes>
```




```
<Figure size 864x576 with 0 Axes>
```



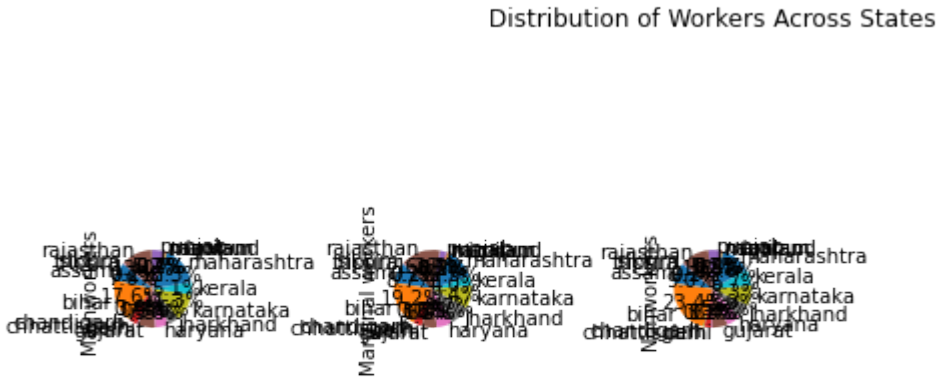
5. Workforce Analysis:

- Pie chart showing the distribution of workers (main workers, marginal workers, non-workers) across states.
- Pie chart showing the distribution of workers across districts within a state.

In [187]:

```
import matplotlib.pyplot as plt
import pandas as pd

# Assuming you have a DataFrame named dfv containing the necessary data

# Calculate the distribution of workers across states
state_worker_distribution = dfv.groupby('state')[['Mainworkers', 'Margi

# Plot a pie chart to visualize the distribution of workers across stat
plt.figure(figsize=(10,8))
state_worker_distribution.plot(kind='pie', subplots=True, autopct='%1.1
plt.title('Distribution of Workers Across States')
plt.axis('equal')
plt.tight_layout()
plt.show()

# Define a function to plot pie chart for worker distribution across di
def plot_district_worker_distribution(state_name):
    # Filter data for the specific state
    state_data = dfv[dfv['state'].str.lower() == state_name.lower()]

    # Calculate the distribution of workers across districts
    district_worker_distribution = state_data.groupby('District')[['Mai

    # Plot a pie chart to visualize the distribution of workers across
    plt.figure(figsize=(10,8))
    district_worker_distribution.plot(kind='pie', subplots=True, autopc
    plt.title(f'Distribution of Workers Across Districts in {state_name
    plt.axis('equal')
    plt.tight_layout()
    plt.show()

# Example usage for a specific state (e.g., Karnataka)
plot_district_worker_distribution('karnataka')

```

```
<Figure size 720x576 with 0 Axes>
```



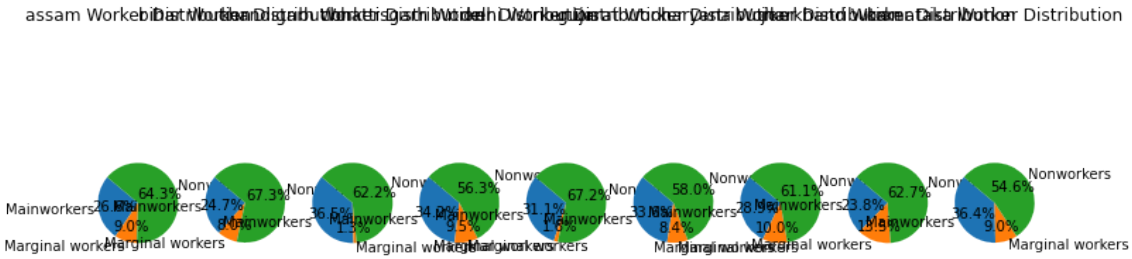

```
<Figure size 720x576 with 0 Axes>
```

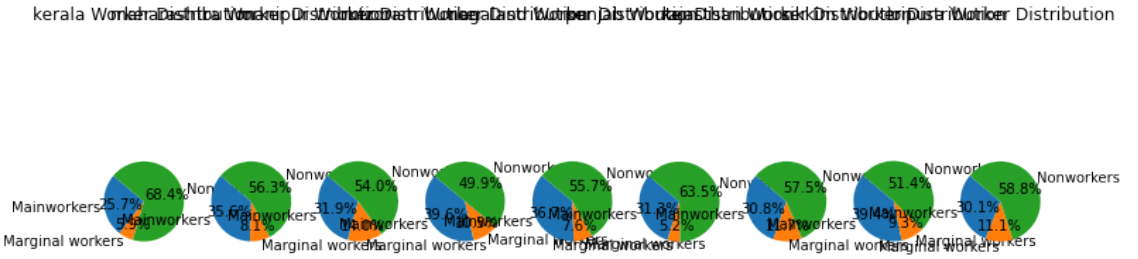

Distribution of Workers Across Districts in Karnataka
Main Workers
Marginal Workers
shimoga
raichur
tumkur
mandya
koppal
belgaum
kolar
bellary
kodagu
hassan
gulbarga
chitradurga
davanagere
dharwad

In [191]:

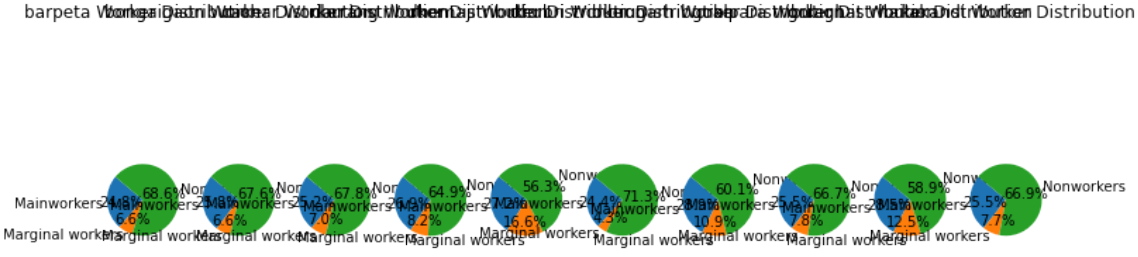
```
import matplotlib.pyplot as plt
import pandas as pd

# Assuming you have a DataFrame named dfv containing the necessary data

# Calculate the distribution of workers across states
state_worker_distribution = dfv.groupby('state')[['Mainworkers', 'Margi

# Plot a pie chart to visualize the distribution of workers across stat
plt.figure(figsize=(12, 10))  # Set the size of the entire figure
num_states = len(state_worker_distribution)

for i, (state, data) in enumerate(state_worker_distribution.iterrows(),
    plt.subplot(2, num_states // 2 + num_states % 2, i)  # Create indiv
    plt.pie(data, labels=data.index, autopct='%1.1f%%', startangle=140)
    plt.title(f'{state} Worker Distribution')
    plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn a

plt.tight_layout()
plt.show()

# Define a function to plot pie chart for worker distribution across di
def plot_district_worker_distribution(state_name):
    # Filter data for the specific state
    state_data = dfv[dfv['state'].str.lower() == state_name.lower()]

    # Calculate the distribution of workers across districts
    district_worker_distribution = state_data.groupby('District')[['Mai

    # Plot a pie chart to visualize the distribution of workers across
    plt.figure(figsize=(12, 10))  # Set the size of the entire figure
    num_districts = len(district_worker_distribution)

    for i, (district, data) in enumerate(district_worker_distribution.i
        plt.subplot(2, num_districts // 2 + num_districts % 2, i)  # Cr
        plt.pie(data, labels=data.index, autopct='%1.1f%%', startangle=
        plt.title(f'{district} Worker Distribution')
        plt.axis('equal')  # Equal aspect ratio ensures that pie is dra

    plt.tight_layout()
    plt.show()

# Example usage for a specific state (e.g., Assam)
plot_district_worker_distribution('Assam')

```

```
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/913872454.py:19: UserWarn
ing: Tight layout not applied. tight_layout cannot make axes width small e
nough to accommodate all axes decorations
  plt.tight_layout()
```






```
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/913872454.py:40: UserWarn
ing: Tight layout not applied. tight_layout cannot make axes width small e
nough to accommodate all axes decorations
  plt.tight_layout()
```



6. Religion Analysis:

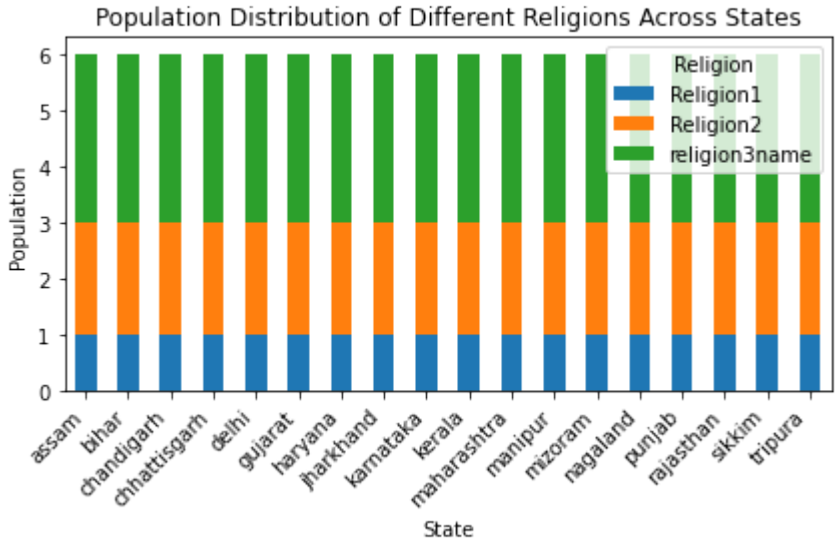
- Stacked bar plot showing the population distribution of different religions across states.
- Stacked bar plot showing the population distribution of different religions across districts within a state.

In [199]:

```
import matplotlib.pyplot as plt
import pandas as pd

# Assuming you have a DataFrame named dfv containing the necessary data
# Extract numeric values from string format
state_religion_distribution[['Religion1', 'Religion2', 'religion3name']

# Plot stacked bar plot for the population distribution of different re
plt.figure(figsize=(12, 8))
state_religion_distribution.plot(kind='bar', stacked=True)
plt.title('Population Distribution of Different Religions Across States
plt.xlabel('State')
plt.ylabel('Population')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Religion')
plt.tight_layout()
plt.show()




import pandas as pd
import matplotlib.pyplot as plt

# Example DataFrame (replace this with your actual DataFrame)
district_religion_distribution = pd.DataFrame({
    'District': ['District1', 'District2', 'District3'],
    'Religion1': ['1.Hindus', '2.Hindus', '3.Hindus'],
    'Religion2': ['4.Hindus', '5.Hindus', '6.Hindus'],
    'religion3name': ['7.Hindus', '8.Hindus', '9.Hindus']
})

# Define a function to extract numeric values from string format
def extract_numeric(s):
    try:
        return float(''.join(filter(str.isdigit, s)))
    except ValueError:
        return None

# Apply the function to the relevant columns
district_religion_distribution[['Religion1', 'Religion2', 'religion3nam

# Plot stacked bar plot for the population distribution of different re
plt.figure(figsize=(12, 8))
district_religion_distribution.plot(kind='bar', stacked=True)
plt.title(f'Population Distribution of Different Religions Across Distr
plt.xlabel('District')
plt.ylabel('Population')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Religion')
plt.tight_layout()
plt.show()


```

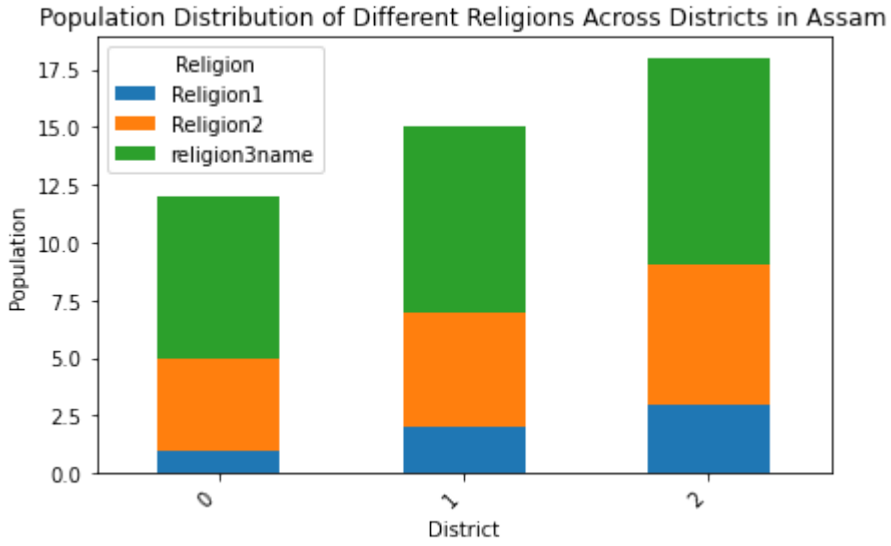
```
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/2184369892.py:6: FutureWa
rning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
  state_religion_distribution[['Religion1', 'Religion2', 'religion3name']]
= state_religion_distribution[['Religion1', 'Religion2', 'religion3nam
e']].applymap(lambda x: int(x.split('.')[0]) if isinstance(x, str) else x)
```

```
<Figure size 864x576 with 0 Axes>
```




```
C:\Users\ritti\AppData\Local\Temp/ipykernel_8252/2184369892.py:41: FutureW
arning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
  district_religion_distribution[['Religion1', 'Religion2', 'religion3nam
e']] = district_religion_distribution[['Religion1', 'Religion2', 'religion
3name']].applymap(extract_numeric)
```

```
<Figure size 864x576 with 0 Axes>
```



In [ ]:
```
1
```

In [ ]:
```
1
```

In [ ]:
```
1
```

In [ ]:
```
1
```

In [ ]:
```
1
```